1. Given the polynomial
$$A(x) = 2 + 7x + 1x^2 + 8x^3$$

   (a) Compute $DFT(A)$ directly by substituting the sample points into the polynomial. (You could multiply $V \cdot A$ if you want.)

   (b) Compute $DFT(A)$ using the FFT algorithm. You should evaluate all recursive calls through the base case (i.e., when the input is a single coefficient).

2. The *convolution* of two vectors A and B of length n is the vector $A * B$ of length n defined by
$$(A * B)[k] = \sum_{i,j:i+j=k} A[i] \cdot B[j]$$
for all $k = 0, 1, \ldots, n-1$.

   (a) Compute the convolution of A (the coefficient vector from above) and $B = (3, 1, 4, 2)$ directly.

   (b) Compute $A(x) \cdot B(x)$. What do you notice about the coefficients of $A(x) \cdot B(x)$ and the convolution $A * B$?

   (c) Give an algorithm to compute in $A * B$ in $O(n \log n)$ time.

3. Suppose you are given an array A of n positive integers. An interval in A is a contiguous subarray $A[i..j]$ for some pair of indices $0 \leq i \leq j < n$. Suppose we compute the sum of elements in each of the $\Theta(n^2)$ intervals in A; some of these sums might coincide. This question asks you to design and analyze efficient algorithms to compute the number of distinct interval sums in A.

   (a) Describe an algorithm that runs in $O(n^2 \log n)$ time.

   (b) Describe an algorithm that runs in $O(M \log M)$ time, where $M = \sum_i A[i]$.

   For example, given the input array $A = (8, 2, 3, 5)$, your algorithms should return 7, because A has seven distinct interval sums: $2, 3, 5, 8, 10, 13$, and 18. For example, A contains two intervals $(8, 2)$ and $(2, 3, 5)$ that both sum to 10.